



EECS 841:

Lecture 14 Classification in Vision

Instructor: Sumaiya Shomaji,
Assistant Professor,
Dept. of EECS, University of Kansas
Email: shomaji@ku.edu
Office: 2020 Eaton Hall and 251 Nichols Hall

Slide Credit: Materials are used from "Introduction to Biometrics" text authors Arun Ross and Anil Jain as well as Dr. Walter Scheirer, Dr. Damon Woodard, and Ming-Hsuan Yang

Introduction

- Classification = labelling “classes” from images/features
- Broadly categorized into three methods:
 - Traditional template-based matching methods, ex. correlation
 - (Traditional) machine learning based methods, ex. PCA, SVM
 - Deep learning methods, ex. CNN, Transformers, Autoencoders etc.

Top Classifiers in Vision

- Convolutional Neural Networks:
AlexNet, ResNet, EfficientNetV2, ConvNeXt etc.
- Vision transformers:
ViT, DeiT, Swin Transformer etc.
- Others: Autoencoders (ex. VAE), Graph Neural Networks (GNN), Generative Adversarial Network (GAN), Hybrid etc.

Top Classifiers in Vision

- Convolutional Neural Networks
- Vision transformers

1. CNNs

Introduction:

- Foundational architecture for computer vision
- Use **convolutional layers** to automatically learn features from input images.

Architecture Summary:

- Convolutional Layers: Apply filters (small matrices) to the input image to extract features.
- Pooling Layers: Reduce spatial size, keeping important information and speeding up computation.
- Fully Connected Layers: Combine learned features to make the final classification.
- Activation Functions: Usually ReLU (Rectified Linear Unit), or Softmax (during classification)

Popular Architecture for Classification in Machine Vision/Computer Vision:

- AlexNet (2012) – the pioneer
- VGG16 (2014) – very deep, simple
- ResNet (2015) – allow even deeper networks skipping connections
- ConvNeXt (2022) – modern, high-performing CNN

1. CNNs_ConvNeXt

Introduction “ConvNeXt”:

- Modern neural network architecture for computer vision = CNN
- ConvNeXt = upgraded CNN + “some” features from transformers
- Target: performance as robust as transformers but speed and simplicity as similar as CNNs
- Originated from ResNet (especially the convolution cores)
- “Some” features from transformer: Layer normalization and Patches based design patterns
 - next slide has details!
 - hints: transformers are popular to handle sequence-based text/language data
 - any guesses how it is relevant to image classification (ex. face data)?

1. CNNs_ConvNeXt

What's New in CNN but already in Transformer:

- Patching: Splitting the image into small square chunks (called patches) before feeding them into the model.

Each patch is like a tiny image (e.g., 4×4 or 16×16 pixels)

Helps the model treat image regions like tokens (inspired by transformers)

- Normalization LayerNorm:

Makes training faster and more stable

It works on **each data sample** independently, unlike BatchNorm

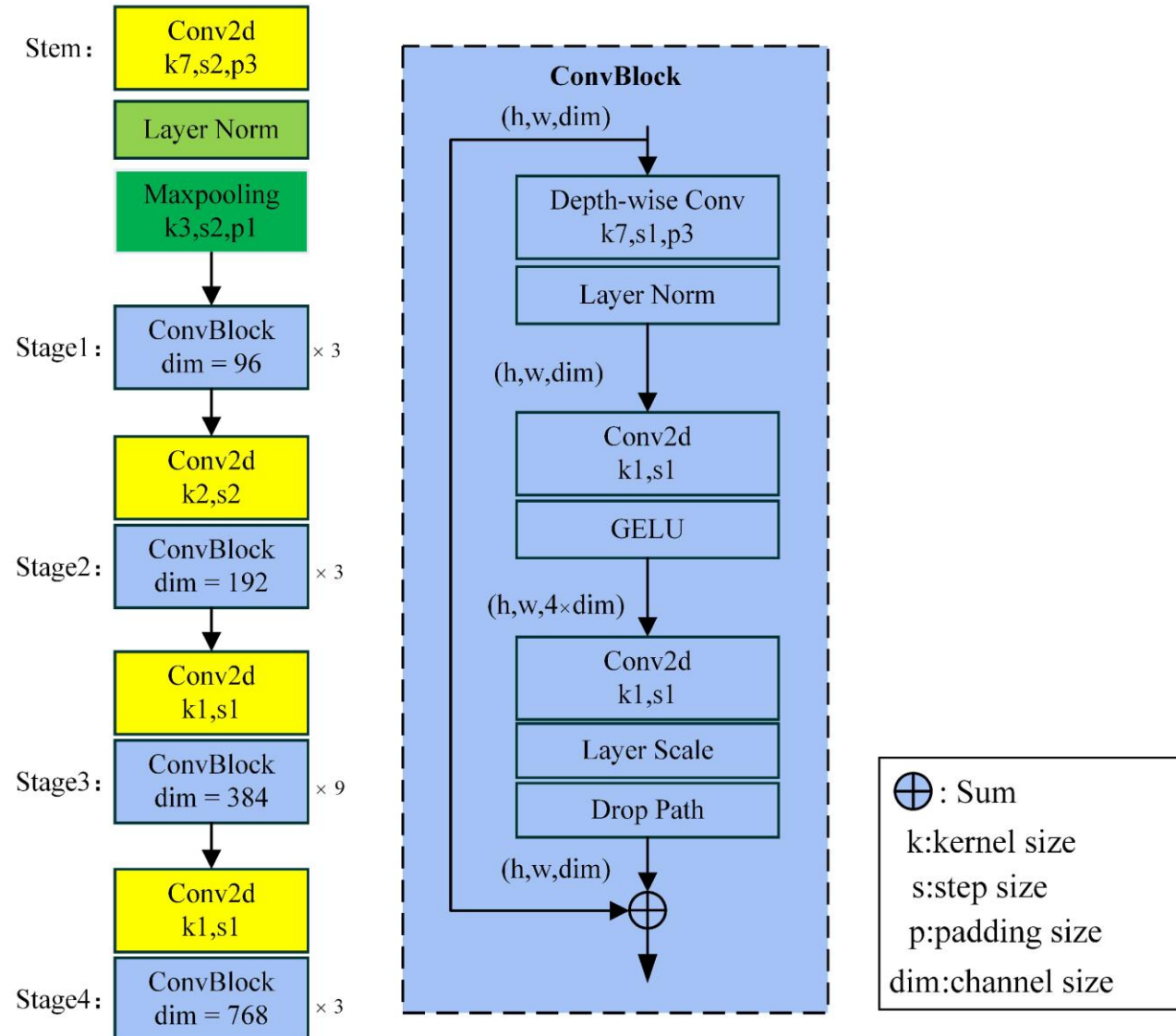
- Actional GeLU: (**Gaussian Error Linear Unit**) is an activation function that works like a smooth version of ReLU.

Allows small negative values to pass through

Gives better performance than ReLU in many modern models like BERT and ConvNeXt

1. CNNs_ConvNeXt

Overall Architecture:
(steps on next slide)



1. CNNs_ConvNeXt

1. Patchify the Image

- image broken into **small square patches**, like tiles
- patches go through a convolutional layer to turn them into feature maps

2. Stacked Blocks (the core layers)

a. Depthwise Convolution

- To extract patterns from each channel of an image → instead of looking at all color channels at once, looks at each one separately → ex. look at only the red parts first, then green, then blue.

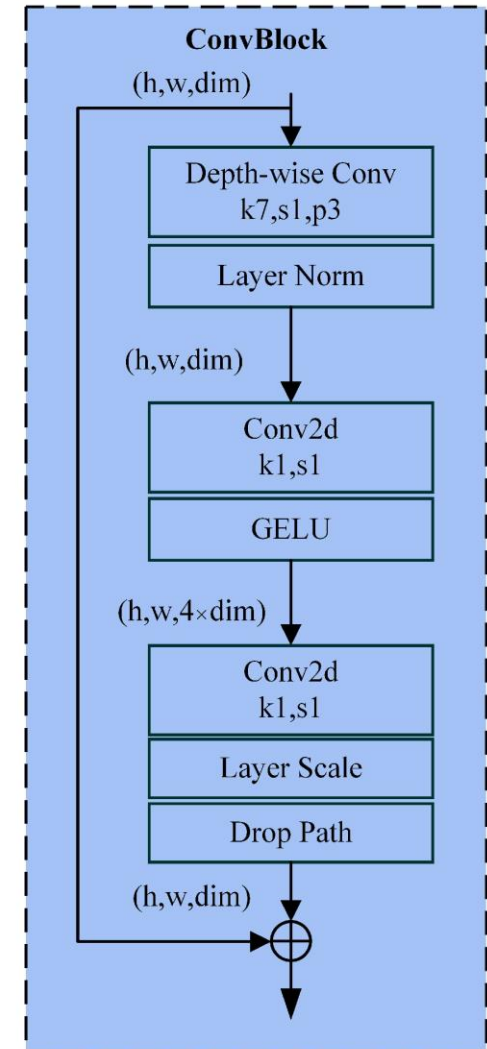
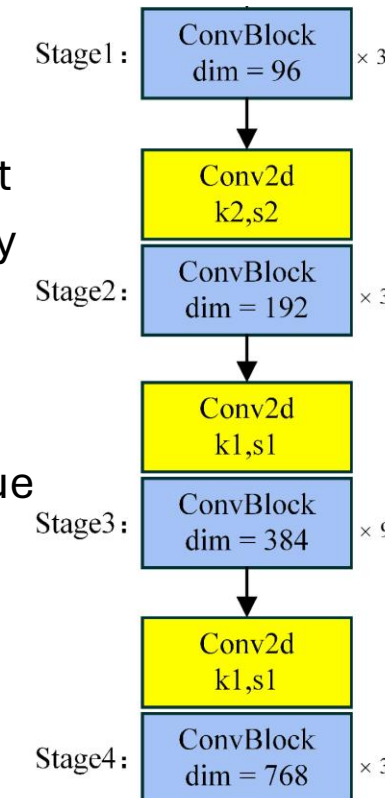
b. Pointwise Convolution (1×1 conv)

- Mixes information from all channels— e.g., putting the red, green, and blue layers back together.

c. Layer Normalization

- This helps stabilize training

d.



1. CNNs_ConvNeXt

1. Patchify the Image

- image broken into **small square patches**, like tiles
- patches go through a convolutional layer to turn them into feature maps

2. Stacked Blocks (the core layers)

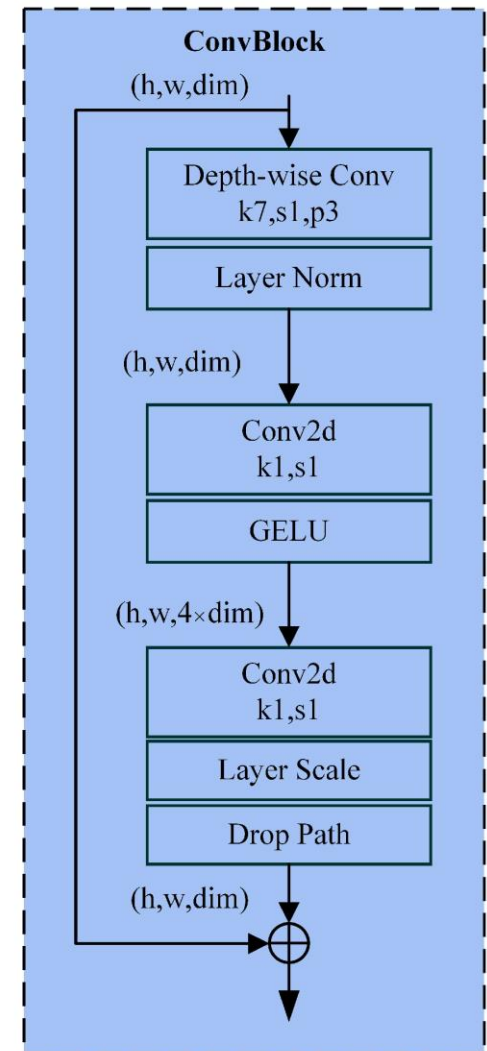
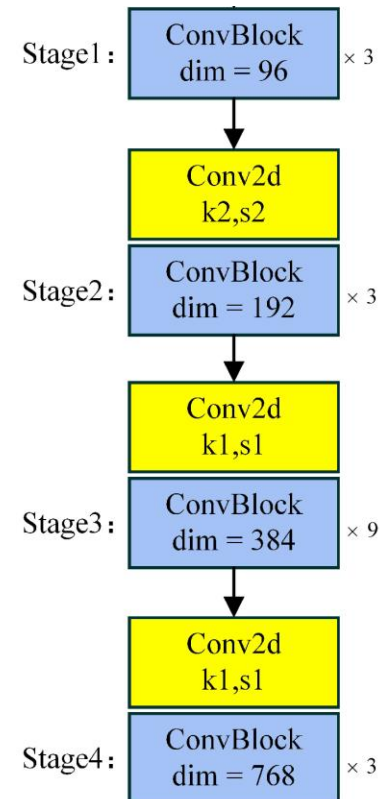
... ..

d. Activation Function (GELU)

- Adds a non-linearity pattern → lets network learn more complex patterns.
- GELU (Gaussian Error Linear Unit) is smoother than ReLU.

e. Residual Connections

- These are like shortcuts → allows the network to remember important features from earlier layers → for ex. *jumping back* to an earlier step to recall a key clue.



1. CNNs_ConvNeXt

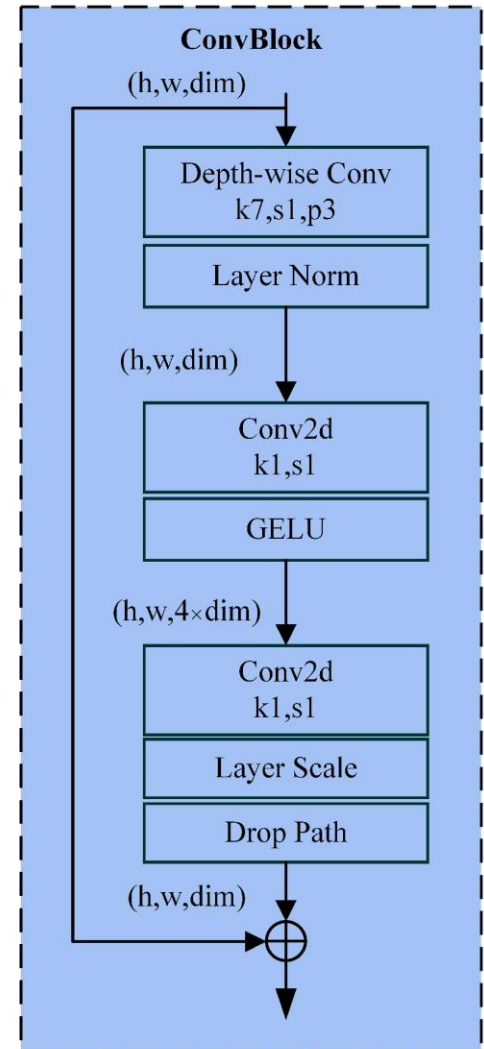
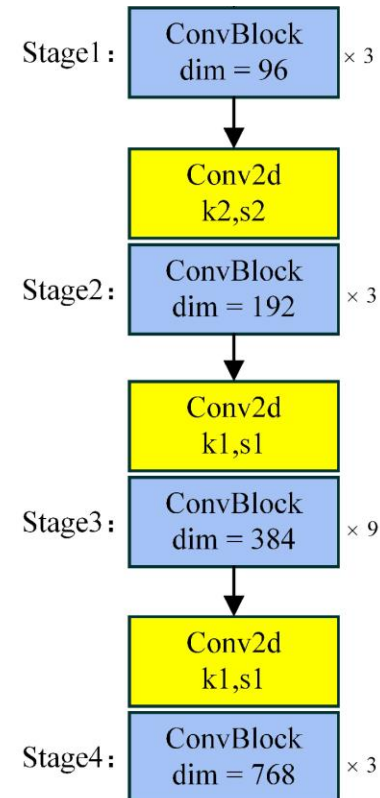
.....

3. Downsampling Layers

- Between groups of blocks, the image size is reduced
- Combination of downsampling but individual channel processing importance helps the model see larger yet abstract parts of the image (first edges, then shapes, then full objects).

4. Final Classifier

- Features are **flattened** and passed through a **fully connected layer** (a decision-maker) to classify the image into a category (like "cat", "dog", etc).



1. CNNs_ConvNeXt

Performance:

- Matches or beats transformer models (like ViT) using only convolutions
- Results on ImageNet benchmark

Model	Top-1 Accuracy
ConvNeXt-Tiny	~82.1%
ConvNeXt-Small	~83.1%
ConvNeXt-Base	~83.8%
ConvNeXt-Large	~84.3%
ConvNeXt-XL	~85.8%

(ImageNet contains over 14 million images and ImageNet-1K has 100 classes)

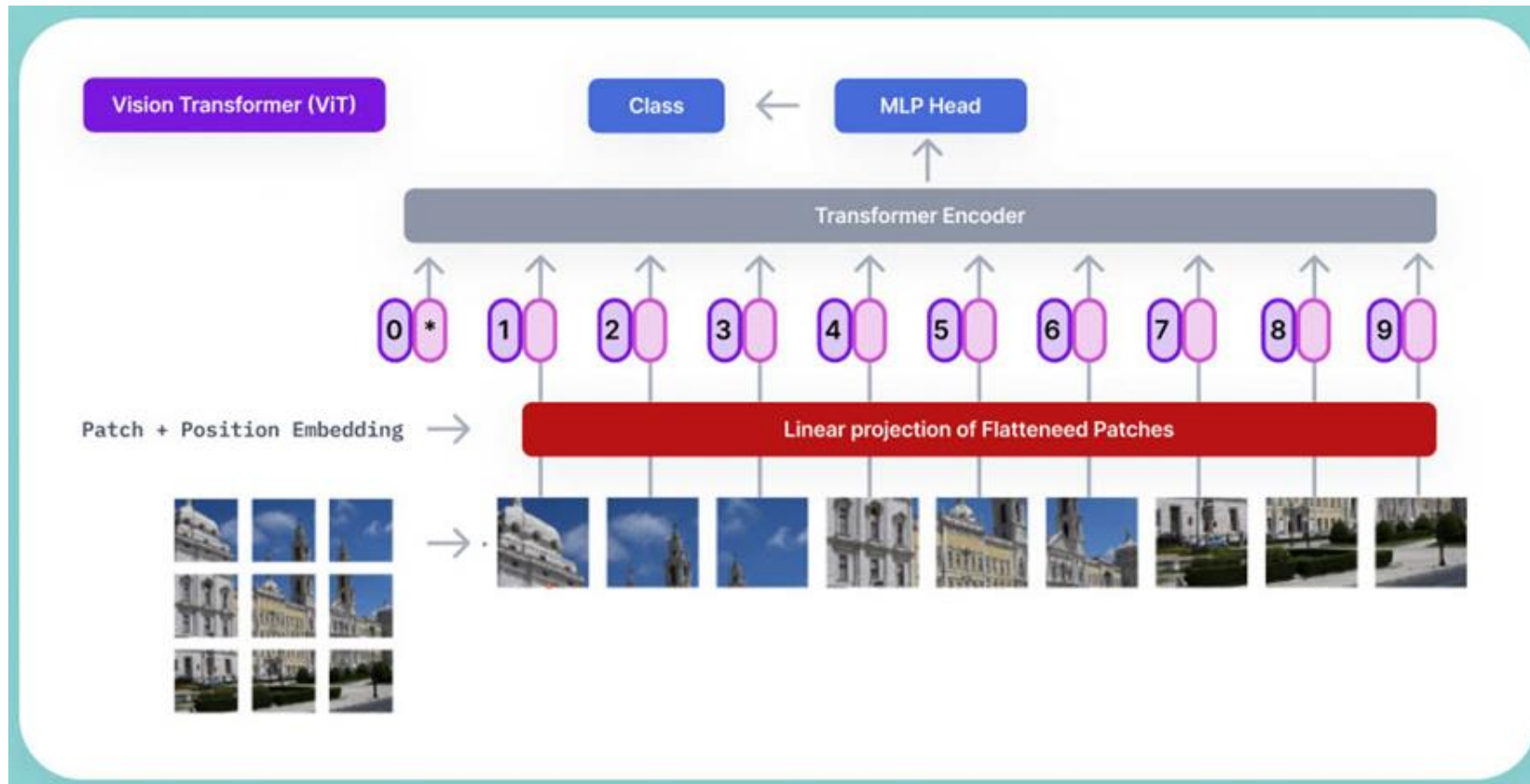
Top Classifiers in Vision

- Convolutional Neural Networks
- Vision transformers

2. Vision Transformers

Introduction:

- State-of-the-art performance on ImageNet for classification in computer vision
- Splits an image into patches → embeds them with positional info → uses self-attention to classify the image



2. Vision Transformers_details architecture

1. Input Image → Split into Patches

2. Patch Embedding

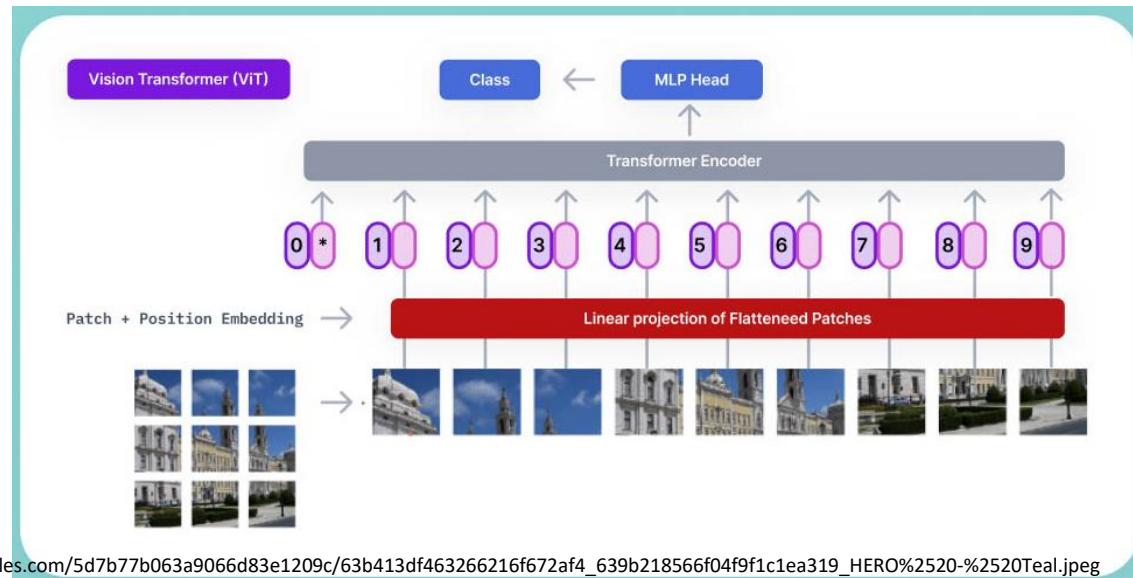
- Each patch passed through a **linear projection layer** = fixed-size vector (e.g., 768 dimensions).

3. Add [CLS] Token

- Classification token → taking notes and summering every imp details from patches and will give the final report

4. Add Positional Embeddings

- Model unaware about order → add **position vectors** to teach the model where patch sits.



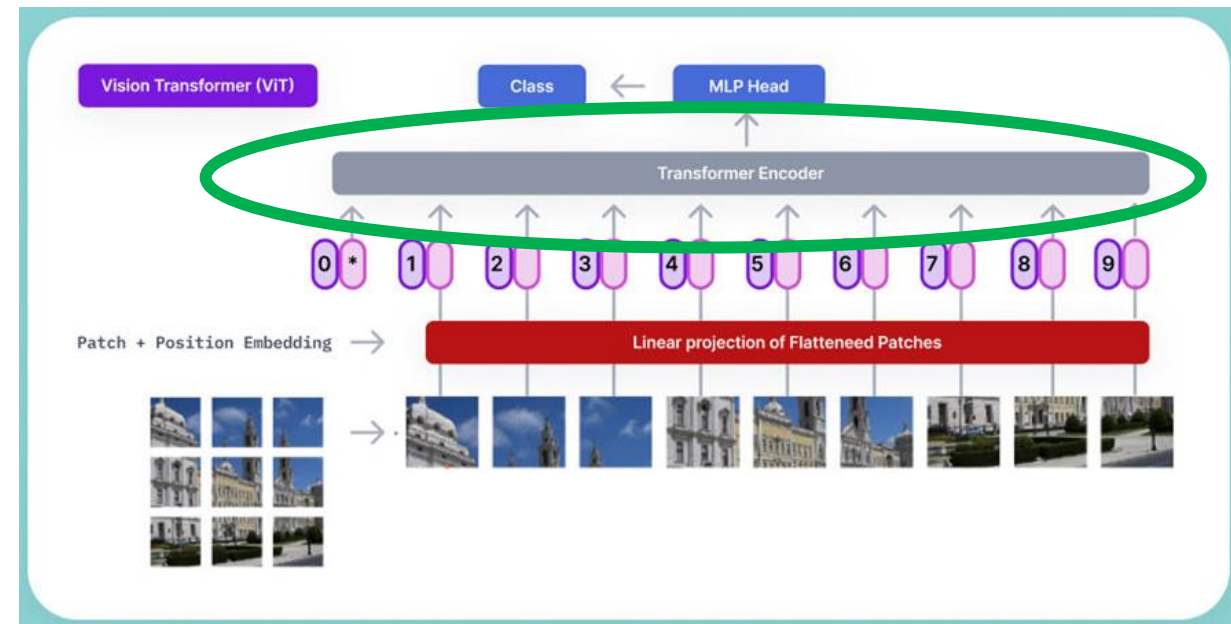
2. Vision Transformers

5. Transformer Encoder Blocks ($\times N$)

- Multi-Head Self-Attention (MHSA): to **help each patch "look at" every other patch** to understand the full image. (attention = learning both the order and pattern)
- Feedforward Neural Network (FFN): Adds non-linearity and depth $\rightarrow$ **mini neural network that processes each patch** token independently after self-attention (ex. token attends other $\rightarrow$ needs a moment to process the updated features $\rightarrow$ FFNs helps)
- Residual Connections + LayerNorm:
Adds **stability** and better gradient flow

How:

- Residual connection: keeps **original input as backup** and adds it to the transformed output in case new one is not better)
- Prevents information loss across layers.
- Solves the vanishing gradient problem (where deep networks stop learning).
- Helps the model learn faster and more accurately.
- LayerNorm **rescales** each input to have a mean of 0 and standard deviation of 1 $\rightarrow$ keeps all layers working on values in a safe range $\rightarrow$ making learning more stable.



2. Vision Transformers

6. [CLS] Token Produces Output

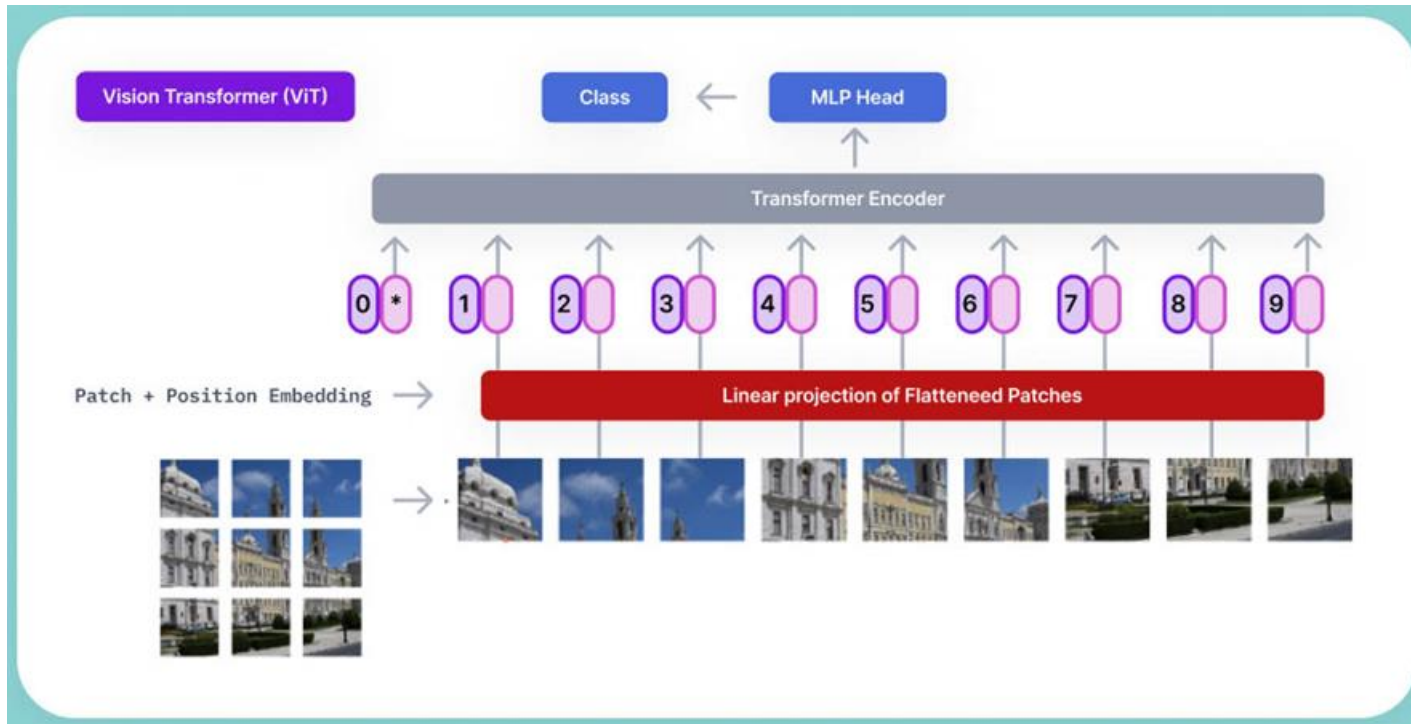
Each After transformer layers, the [CLS] token holds the image's "summary"

7. MLP Head → Classification

- Final [CLS] vector goes through a classifier to output class probabilities

Performance:

*"The CSWin Transformer, a variant of ViT, achieved a top-1 accuracy of **85.4%** on ImageNet-1K. Another study reported that Swin Transformers beat the state-of-the-art result with a validation accuracy of **91.35%** on Tiny ImageNet."*



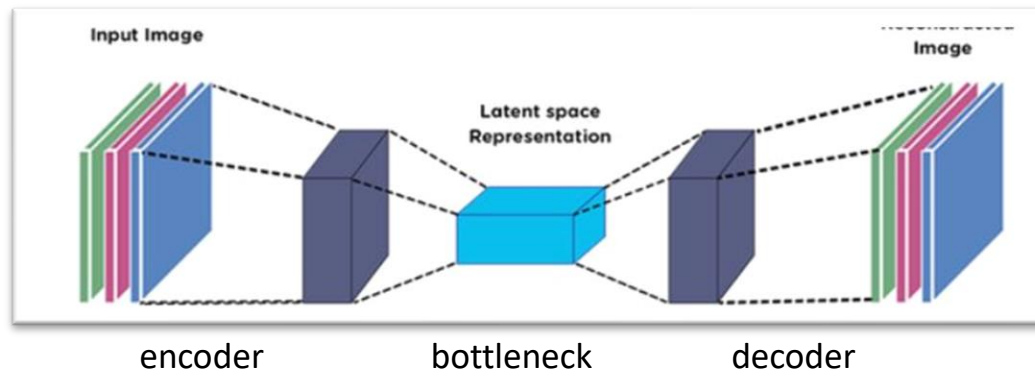
Other Classifiers in Vision

- Autoencoder classifier
- GNN classifier

Auto Encoders

Introduction:

- Designed to learn efficient representations of input data
- Main purpose: dimensionality reduction or feature learning



- Train an autoencoder on input images to learn good latent representations = compressed features
- Discard the decoder (when loss function is minimized)
- Feed the encoder output (= latent vector) to a separate classifier (e.g., fully connected neural network, SVM, etc.) for classification.

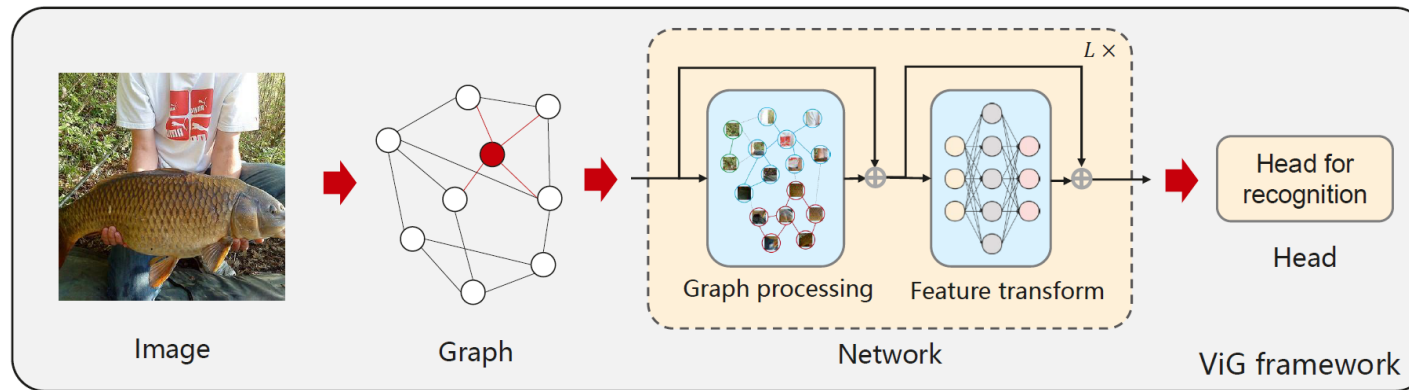
Application in Computer Vision:

- **Feature Extraction for Classification:** compresses input images into a latent space → captures essential features → compressed representations serve as inputs to classifiers → improves accuracy in classification
- **Denoising for Classification:** Clean noisy images → producing clearer inputs for classifiers → enhancing classification performance.
- **Dimensionality Reduction:** reduces the dimensionality of image data → alleviates curse of dimensionality → more efficient classification.

Graph Neural Networks

Introduction:

- Designed to learn relationship between objects
- How: data converted to graph structure → graph neural network learns the relationship → learn that pattern → useful for classification



Ref: https://aisholar.s3.ap-northeast-1.amazonaws.com/posts/May2023/VisionGNN_%E5%9B%B3%E5%BC%92.png

Vision GNN (ViG) Architecture:

1. Grapher Module: uses graph convolution to aggregate and update graph information
2. FFN Module: a feedforward neural network with two fully connected layers for node feature transformation.

Cluster Vision GNN (ClusterViG) Architecture:

1. Partitions the input image → dynamic efficient graph convolution (DEGC) to improve graph inference
2. Reduced inference latency up to 5X while keeping up the state-of-the-art classification performance of ViG

- Issues with learning guided Vision classification?
- Possible solution

- **Issues with learning guided Vision classification?**
 - time consumption
 - need a large training sample
 - prone to model attack
- **Possible solution?**
 - Simpler models
 - Feature extractors
 - Probabilistic data structures

Examples of Solutions

Feature Extraction:

- HOG, SIFT, color histograms, etc.

“Simple” Classification: Support Vector Machine (SVM), k-Nearest Neighbors (k-NN), Random Forest, Logistic Regression, Naïve Bayes, Decision Tree, Linear Discriminant Analysis (LDA)

Probabilistic Structures:

- No learning at all, yet speedy comparison of distinct features:
 - Bloom Filter
 - Hierarchical Bloom Filter

Feature Extractor: Histogram of Gradients (HoG)

1. Convert the image to **grayscale**
2. Compute **gradients** at each pixel at x,y index:

$$G_x = I(x + 1, y) - I(x - 1, y), \quad G_y = I(x, y + 1) - I(x, y - 1)$$

(gradients showing how the brightness is changing in x and y directions.)

3. Compute gradient **magnitude** and **direction**:

$$\text{Magnitude} = \sqrt{G_x^2 + G_y^2}, \quad \text{Angle} = \arctan\left(\frac{G_y}{G_x}\right)$$

4. Divide the image into small **cells** (like 8x8 pixels).
5. Create a **histogram** of gradient directions in each **cell**: (e.g., how many edges point 0°, 20°, ..., 160°?)
6. Divide **whole image in blocks** containing (e.g., 2x2 cells)
7. **Normalize** across **blocks** to reduce lighting effects.
8. **Concatenate** all normalized histograms as Final Full Vector

Classifier: Support Vector Machine

SVM tries to find the best **hyperplane** that separates two classes. The decision function for a linear SVM is:

$$f(x) = w^T x + b$$

Where:

- x = input feature vector (e.g., image features)
- w = weight vector (learned by SVM)
- b = bias term

Steps:

1. Take images $\rightarrow$ raw/ features
2. Train test split
3. Find the hyperplane to maximize the margin between classes

classification rule is:

If $f(x) \geq 0$, predict class $+ 1$

If $f(x) < 0$, predict class $- 1$

Secure Matching System in Computer Vision:

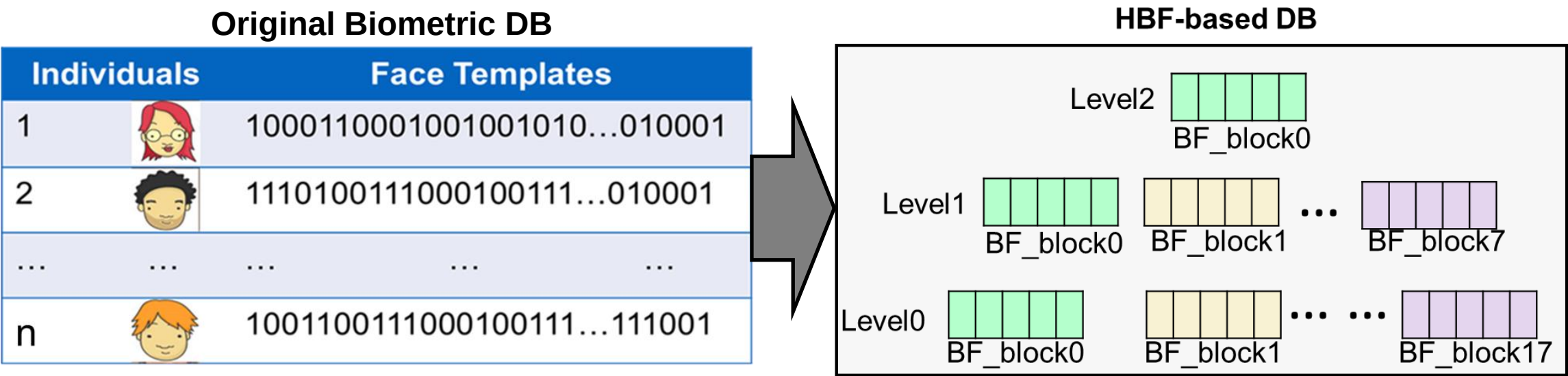
Background:

- Current studies only solve the problems individually

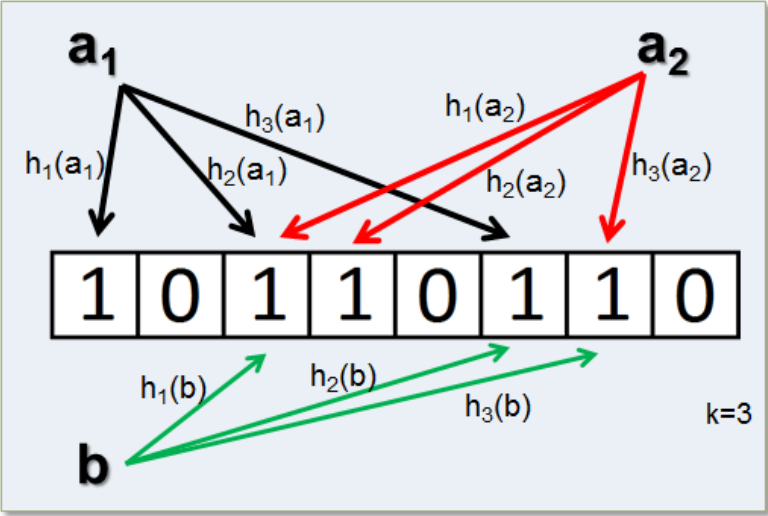
Method	Advantage	Weakness
CNN	Noise-handling	Large storage, Slow query, Privacy
kNN	Less storage, Fast query	Noise sensitive, Privacy

Why Hierarchical Bloom Filter:

- Hierarchical Bloom Filter for data structure for Security, Space-efficiency, Rapid Query, and Noise Handling
- **First study** that solves all four problems!



Secure Matching System: Bloom Filter



$m = -\frac{n \ln p}{\ln^2 2}$	m – filter size (bits) p – error probability (false positive) n – maximum cardinality (capacity)
$k = \frac{m}{n} \ln 2$	k – number of hash functions m – filter size (bits) n – maximum cardinality (capacity)

BF-based Classification System

Bloom Filter (BF):

- BF probabilistic data structure (BF is like a binary vector)
- 2 step operation: enrollment and membership query processing

Original
Biometric DB

Individuals		Face Templates
1		1000110001001001010...010001
2		1110100111000100111...010001
...	...	...
n		1001100111000100111...111001



BF-based
Biometric DB

1	1	0	1	0	1	0	1	0	0	1	1	0	0	1	...	0	0	1	0
1	2	3	4	5	6	7	8	9	.	.	.	.	.	.	.	.	.	.	m

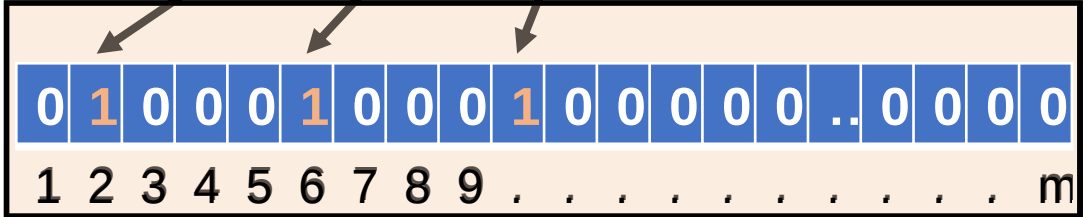
Enrollment: Feature Extraction and Storage in BF:

Original DB

Individuals		Face Templates
1		1000110001001001010...010001
2		1110100111000100111...010001
...	...	...
30000		1001100111000100111...111001

Hash
(k=3)

Output = 2,6,10

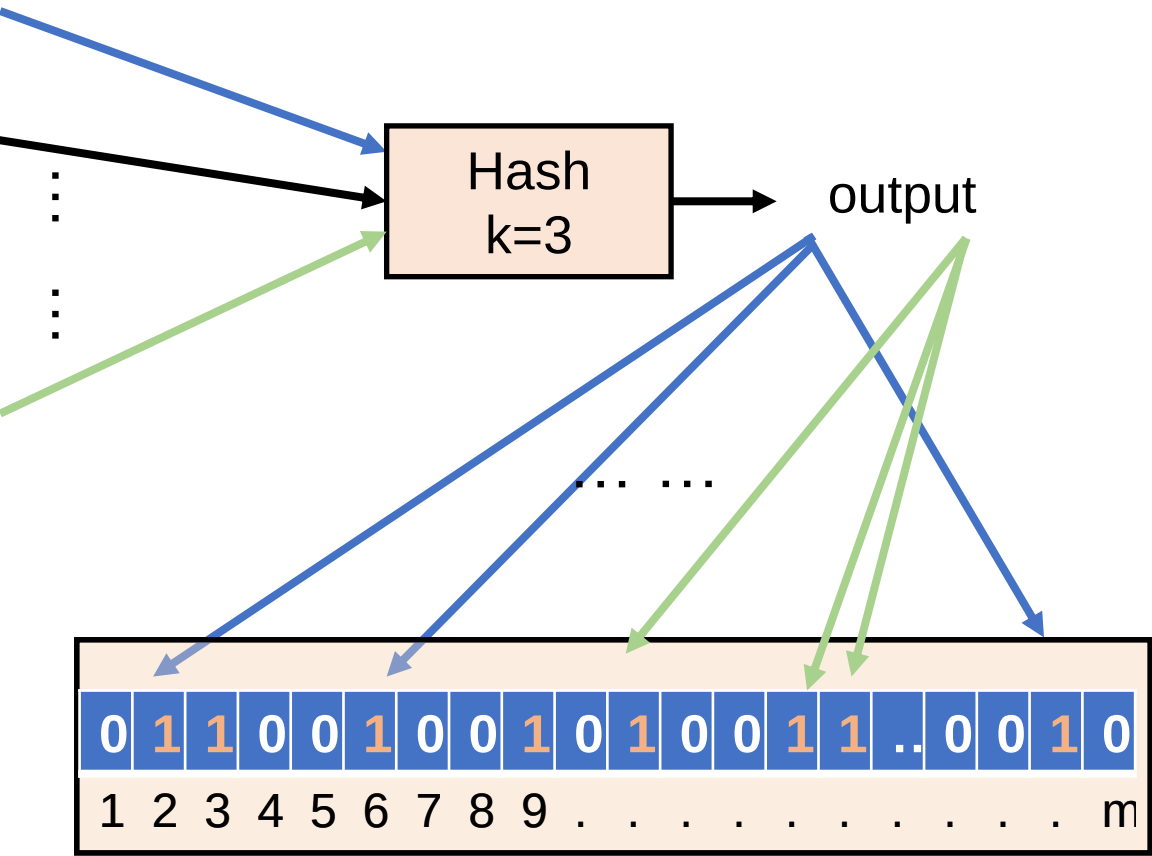


Target: BF database

Enrollment: Feature Extraction and Storage in BF:

Original DB

Individuals		Face Templates
1		1000110001001001010...010001
2		1110100111000100111...010001
...	...	...
30000		1001100111000100111...111001



Target: BF database

Query in BF for Classification:

Phase 1: Enrollment

Original DB		
Individuals		Face Templates
1		1000110001001001010...010001
2		1110100111000100111...010001
...	...	...
30000		1001100111000100111...111001

BF DB (after enrollment)

0	1	1	0	0	1	0	0	1	0	1	0	0	1	1	..	0	0	1	0
1	2	3	4	5	6	7	8	9	.	.	.	.	.	.	.	.	.	.	m

Phase 2: Query



Hash
(k=3)

Output = 1,5,9

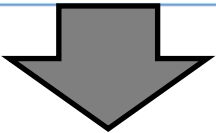


Challenge:
• Noise

Solution: HBF-based Biometric System

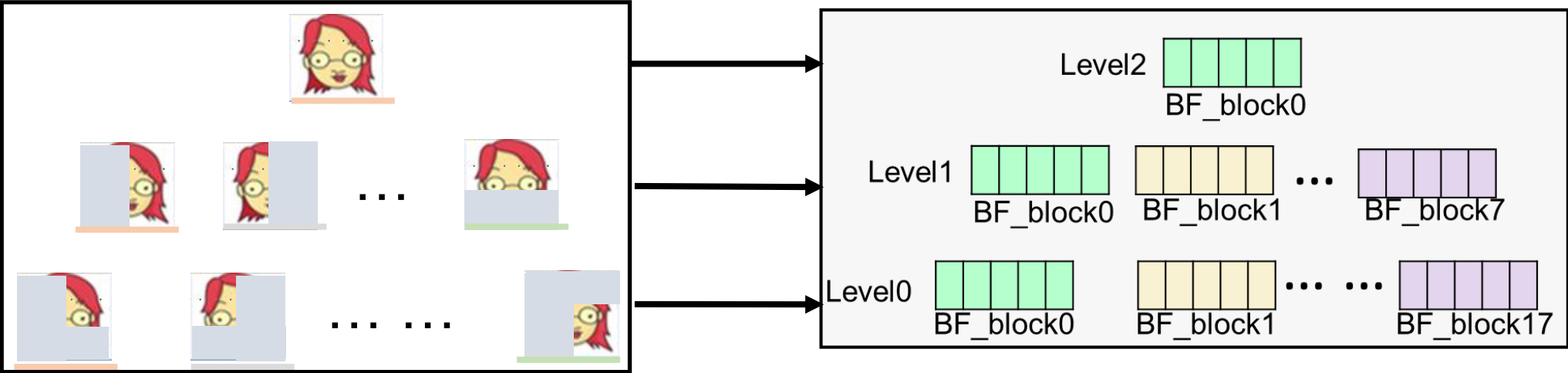
Original Biometric DB

Individuals		Face Templates
1		1000110001001001010...010001
2		1110100111000100111...010001
...	...	...
n		1001100111000100111...111001



Segmenting templates for segment-wise enrollment

HBF-based DB



Performance Evaluation for Face Recognition Case Study:

Authentication Performance In Presence of Noise:

Metric	Legacy BF	Proposed HBF
FPR	0.13	0
FNR	0.42	0
TPR	0.57	1
TNR	0.86	1

Summary:

- **Space-efficiency:** 12x storage reduction
- **Rapid operation:** 570x faster search speed
- **Security:** Security and privacy preserved (due to transformation)
- **Noise Tolerance:** Robust to noise (0% false match and 0% false mis-match)

Resource Evaluation:

Metric	Database Type		
	Original	BF based	Proposed HBF based
Query time	14.8s	0.00058s	0.026s
Storage	37.5MB	0.0175MB	2.89MB

HBF is a Promising Solution for any Large-scale Image Classification System “if data is well pre-processed (e.g., scale-rotation corrected)” (especially in Resource resource-constrained environment. 33